

...then a PHP Web page program *login.php* may be written to exploit the SQL injection loophole of the SQL query. The purpose of this code is to read the user's name from the users' table of the database. This code has four parts. Initially, it establishes a connection to a MySQL database *customer*; then it reads the variable user name from the *login.html* Web form and performs the first query to read a particular user's name from the database table users. Subsequently, this section of the code displays all the records with the user name = *\$uname*. The third section performs another query to display the same user's name but with restriction status. In a normal situation, this section will display all records having the user name = *\$uname*, provided the record satisfies the condition *status=1*. Lastly, it closes the database connection and returns the control to the system.

Step 1: Open a database customer.

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "customer";
// Create a connection
$conn = new mysqli($servername, $username, $password,
$dbname);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
else{
echo "Connected successfully<br>";
}
```

Step 2: Read the POST variable user name from the Web form and perform the SQL select query.

```
$uname = $_POST['username'];
#1. Read unrestricted records

$sql = "SELECT * FROM users WHERE username = '". $uname;
$result = mysqli_query($conn,$sql);

if ($result->num_rows > 0) {
    // output data of each row
    while($row = $result->fetch_assoc()) {
        echo "<br>id: " . $row["userid"]. " - Name: " .
        $row["username"]. " " . $row["status"]. "<br>";
    }
}
else {
    echo "0 results <br>";
}
```

Step 3: Perform the select query with restriction status. Field status will allow only those records that have *status=1*.

```
# 2 Read restricted records
$sql = "SELECT * FROM users WHERE username = '". $uname. "
and status = 1";

$result = mysqli_query($conn,$sql);

if ($result->num_rows > 0) {
    // output data of each row
    while($row = $result->fetch_assoc()) {
        echo "<br>id: " . $row["userid"]. " - Name: " .
        $row["username"]. " " . $row["status"]. "<br>";
    }
} else {
    echo "0 results";
}
```

Step 4: Close the connection and the PHP Web page *login.php*.

```
$conn->close();
?>
```

With this code snippet, one can use the given Web based form to exploit the SQL injection technique to bypass the user name verification clause of the SQL query statement. The query statement:

```
$sql = "SELECT * FROM users WHERE username = '". $uname;
```

...can inject a string into the statement to browse the entire table and display all the records of the table. To make the query TRUE for all the situations one can enter the string '*or 1=1*' (Figure 2) instead of a valid user's name. Since *1=1* is always true and the connecting logical operator is OR, it will make the query TRUE for all the records and the subsequent *for-loop* will fetch all the records from the table users (Figure 3). If proper precaution is not taken, this type of injection can make the database vulnerable to hackers.

Retrieving restricted data: Web page mode

To retrieve restricted records as shown in the second query, it is required to bypass the restriction *status = 1* clause of the query statement. This can be done by commenting the restriction with an SQL injection statement *parker '--* (Figure 4), where '*parker*' is a valid user's name. String '*--*' will make the rest of the line commented and the query will display all records with the user name = *parker* (Figure 5).